Example Exam Question — Ridge Regression

Dataset: `data/hitters.csv`
Topic: Ridge Regression with Cross-Validation

Question 1 (25 marks)

The `hitters.csv` dataset contains statistics for 322 Major League Baseball players along with their `Salary` (in thousands of dollars). You are tasked with building a Ridge Regression model to predict player salary.

The dataset has the following columns (among others):
`AtBat`, `Hits`, `HmRun`, `Runs`, `RBI`, `Walks`, `Years`, `CAtBat`, `CHits`, `CHmRun`, `CRuns`, `CRBI`, `CWalks`, `PutOuts`, `Assists`, `Errors`, `Salary`, `League`, `Division`, `NewLeague`

Part (a) — Data Preparation (5 marks)

Load the dataset and prepare it for modelling. Your answer should:

1. Load `data/hitters.csv` and drop rows with missing values.
2. Set `Salary` as the target variable `y`.
3. Drop `Salary`, `League`, `Division`, and `NewLeague` from the features and cast to `float64` to get `X`.
4. Print the shape of `X` and `y`.

Write the code below:

```
# Your answer here
```

Model answer (a):

```
import pandas as pd
import numpy as np

df = pd.read_csv('data/hitters.csv').dropna().drop('Unnamed: 0', axis=1)

y = df['Salary']
X = df.drop(['Salary', 'League', 'Division', 'NewLeague'],
axis=1).astype('float64')

print(X.shape, y.shape)   # (263, 16) (263,)
```

Part (b) — Fit Ridge Regression with Cross-Validation (10 marks)

Using `RidgeCV` from `sklearn`:

1. Standardise `X` by dividing each column by its standard deviation (do **not** use `StandardScaler` — divide manually using `.std()`).
2. Fit a `RidgeCV` model with `alphas` ranging from `0.01` to `100` (1000 evenly-spaced values) and `cv=10`.
3. Print the best alpha chosen by cross-validation.
4. Print the model coefficients and identify which **two** features have the largest absolute coefficients.

Write the code below:

```
# Your answer here
```

Model answer (b):

```
from sklearn.linear_model import RidgeCV

X_std = X / X.std()

rcv = RidgeCV(alphas=np.linspace(0.01, 100, 1000), cv=10)
rcv.fit(X_std, y)

print("Best alpha:", rcv.alpha_)

coef_series = pd.Series(rcv.coef_, index=X.columns).sort_values(key=abs,
ascending=False)
print(coef_series)
print("\nTop 2 features:", coef_series.index[:2].tolist())
```

Part (c) — Evaluate and Interpret (10 marks)

1. Split the standardised data into 70% training and 30% test sets using `random_state=42`.
2. Refit a `Ridge` model (not `RidgeCV`) on the training set using the best alpha found in part (b).
3. Compute the **Mean Squared Error (MSE)** on the test set.
4. In **2–3 sentences**, explain what the Ridge penalty does to the model coefficients and why this is useful when predictors are correlated.

Write the code and answer below:

```
# Your answer here
```

Model answer (c):

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X_std = X / X.std()

X_train, X_test, y_train, y_test = train_test_split(X_std, y, test_size=0.3,
random_state=42)

ridge = Ridge(alpha=rcv.alpha_)
ridge.fit(X_train, y_train)

mse = mean_squared_error(y_test, ridge.predict(X_test))
print(f"Test MSE: {mse:.2f}")
```

Written answer:

Ridge regression adds an L2 penalty term ($\lambda \times \Sigma\beta^2$) to the loss function, which shrinks all coefficients towards zero but never sets them exactly to zero. This reduces variance at the cost of a small increase in bias. It is particularly useful when predictors are correlated (multicollinearity), as ordinary least squares becomes unstable and produces large, unreliable coefficients — Ridge stabilises these estimates by distributing the effect across correlated predictors.

End of Question 1